Cyberstart: dokumentacja techniczna

Dla osób, które chcą dopisać nową grę albo zrozumieć, jak działa aplikacja.
Moduł gry: `js/games/<id>.js`. Wspólne komponenty: `js/ui.js`, `js/terminal-engine.js`, `js/config-audit.js`, `js/log-viewer.js`, `js/scenario.js`. Stan: `js/progress.js`.

```
npm start              http://localhost:4180   (serwer bez zależności,
scripts/serve.mjs)
npm run test:e2e       testy Playwright, port 4181
```

Spis treści

1. Architektura i routing
2. Konwencje treści i kodu
3. Moduł gry
4. Obiekt ctx
5. js/ui.js: budowanie widoku
6. choiceQuestion · runQuiz · draw
7. createTerminal
8. mountConfigAudit
9. logViewer
10. playScenario
11. examPool i egzamin
12. Zapis postępu (localStorage)
13. Eksport i import postępu
14. Odznaki i materiały
15. PWA i service worker
16. Testy e2e

Architektura i routing

Cyberstart to statyczna aplikacja: HTML, CSS i moduły ES, bez frameworka i bez kroku budowania. Nie ma backendu, kont ani analityki. Wszystko, co robi użytkownik, zostaje w jego przeglądarce.

```
index.html ──► js/app.js      router, mapa, widok gry, wynik
                 │
                 ├──► js/games/index.js   rejestr gier (kolejność = ścieżka)
                 │       └──► js/games/<id>.js   jedna gra = jeden moduł
                 │              └──► ui, terminal-engine, config-audit,
                 │                   log-viewer, scenario
                 ├──► js/progress.js      stan w localStorage
                 ├──► js/badges.js        odznaki
                 └──► js/resources.js     „Kolejny krok w praktyce”
sw.js ──► offline: najpierw sieć, potem pamięć podręczna
```

Routing działa na hashu adresu, więc aplikacja działa na każdym hostingu statycznym bez konfiguracji przekierowań:

Adres	Widok
`#/`	Mapa nauki: hero, sekcja „Dziś” (wyzwanie dnia, seria, odznaki), kategorie z grami, lista odznak.
`#/gra/{id}`	Gra: nagłówek, przełącznik poziomu (jeśli gra go ma), „minuta teorii”, pasek postępu, plansza gry.
`#mapa`, `#odznaki`	Przewinięcie mapy do sekcji, bez zmiany widoku.
każdy inny	Widok 404 w aplikacji. Plik `404.html` obsługuje błędne ścieżki po stronie serwera.

Przy zmianie trasy aplikacja przerywa nasłuch klawiatury poprzedniej gry (`AbortController`) i przenosi fokus na `<main>` dla czytników ekranu. Przy pierwszym wczytaniu fokus zostaje na początku strony, żeby link „Przejdź do treści” był pierwszy pod klawiszem Tab.

Konwencje treści i kodu

Zasada	Opis
Język	Całe UI po polsku. Formy neutralne płciowo: zamiast „grałeś" piszemy „Dzisiejsza gra zaliczona", zamiast „sam dzwonię" piszemy „samodzielnie dzwonię".
Liczebniki	Zawsze przez `plural(n, 'gra', 'gry', 'gier')`. Nigdy „34 minigier".
Interpunkcja	Separator to zwykły łącznik. Bez pauz (em dash) i bez średnika jako pauzy w zdaniu.
HTML w treściach	Pola `lesson`, `takeaways`, `why`, `prompt`, `label` to statyczne, autorskie teksty i mogą zawierać `<code>` czy `<strong>`. Tekst wpisany przez użytkownika trafia do DOM wyłącznie przez `textContent`.
Fikcja	Marki są fikcyjne (Bank Horyzont, Kurier Szybko). Adresy IP pochodzą z zakresów dokumentacyjnych `192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24` lub prywatnych.
Charakter treści	Obronny i koncepcyjny: rozpoznawanie ataków, konfiguracja zabezpieczeń, etyka i prawo. Bez instrukcji atakowania cudzych systemów.
Style	Kolory, odstępy, promienie i czasy animacji tylko z tokenów w `css/tokens.css`. Kontrast co najmniej WCAG AA w obu motywach.
Kod	Bez komentarzy w kodzie. Nazwy mówią, co robią. Bez martwego kodu.

Moduł gry

MODUŁ `js/games/<id>.js`

Każda gra eksportuje domyślnie jeden obiekt. Aplikacja na jego podstawie rysuje kartę na mapie, nagłówek gry, lekcję i ekran wyniku, a samą rozgrywkę oddaje funkcji `mount`.

Pola obiektu

Pole	Wymagane	Opis
`id`	tak	Identyfikator w adresie `#/gra/{id}`: małe litery, cyfry i łączniki. Nie zmieniaj go po publikacji, bo od niego zależy zapisany postęp.
`title`	tak	Tytuł gry, używany też w `<title>` strony.
`category`	tak	Dział na mapie nauki: `linux` — Linux i Kali `sieci` — Jak działa sieć `hasla` — Hasła i kryptografia `ludzie` — Ludzie i internet `obrona` — Obrona w praktyce `wiedza` — Powtórka i egzamin
`icon`	tak	Nazwa ikony z `js/icons.js`, np. `terminal`, `shieldCheck`.
`minutes`	tak	Szacowany czas gry w minutach.
`summary`	tak	Jedno lub dwa zdania. Karta na mapie, nagłówek gry i meta description.
`lesson`	tak	Tablica akapitów „Zanim zaczniesz: minuta teorii”.
`takeaways`	tak	Tablica punktów „Zapamiętaj” na ekranie wyniku.
`mount(stage, ctx)`	tak	Rysuje grę w elemencie `stage`. Może być `async`. Kończy się wywołaniem `ctx.finish`.
`levels`	nie	`true` włącza przełącznik poziomu (podstawowy, zaawansowany). Wybrany poziom trafia do `ctx.level`.
`badge()`	nie	Zwraca krótki tekst albo `null`. Mapa pokazuje go na karcie z kropką uwagi, np. „5 do powtórki”.

Przykład: najmniejsza gra z quizem

```
import { runQuiz, draw } from '../ui.js';

const POOL = [
  {
    prompt: 'Co oznacza otwarty port?',
    options: [
      { label: 'Program na nim nasłuchuje', correct: true, why: 'Każdy otwarty port
to drzwi do pilnowania.' },
      { label: 'Kabel jest odłączony', why: 'Port to pojęcie logiczne.' }
    ]
  }
];

export default {
  id: 'przyklad',
  title: 'Przykładowa gra',
  category: 'sieci',
  icon: 'plug',
  minutes: 5,
  levels: true,
  summary: 'Krótki quiz o portach.',
  lesson: ['Port to numer drzwi, za którymi czeka program.'],
  takeaways: ['Zamykaj porty, których nie potrzebujesz.'],
  async mount(stage, ctx) {
    const questions = draw(POOL, 5, ctx.level);
    const score = await runQuiz(stage, ctx, questions);
    ctx.finish({ score, max: questions.length });
  }
};
```

Jak dodać grę

Krok	Co zrobić
1	Utwórz `js/games/<id>.js` według kontraktu powyżej.
2	Dodaj import i wpis w tablicy `GAMES` w `js/games/index.js`. Kolejność w tablicy to kolejność na ścieżce nauki.
3	Style dopisz do `css/games.css`, używając tokenów.
4	Jeśli mechanika jest nietypowa, dodaj jej obsługę do autoplayera w `tests/e2e/helpers.js` (funkcja `special()`) albo oznacz przycisk atrybutem `data-autoplay`.
5	Uruchom `npm run test:e2e`. Nowa gra zostanie automatycznie przetestowana: przejście, telefon, oba motywy, dostępność.

Obiekt ctx

OBIEKT `ctx`

Drugi argument `mount`. Łączy grę z resztą aplikacji: paskiem postępu, zapisem wyniku i cyklem życia widoku.

Znaczenie kluczy

Klucz	Opis
`ctx.level`	`'podstawowy'` albo `'zaawansowany'`. Dla gier bez `levels` zawsze `'podstawowy'`.
`ctx.signal`	`AbortSignal` przerywany przy opuszczeniu gry. Przekazuj go do `onKey`, żeby skróty klawiszowe nie działały po zmianie widoku.
`ctx.stage`	Ten sam element, który dostaje `mount`.
`ctx.progress(zrobione, wszystkie)`	Aktualizuje pasek postępu i licznik, np. `3/12`.
`ctx.finish({ score, max })`	Zapisuje wynik i rysuje ekran wyniku. XP = `round(100 × score / max)`, zapamiętywany jest najlepszy wynik. Przy okazji: seria dni, wyzwanie dnia, nowe odznaki, lista „Zapamiętaj" i „Kolejny krok w praktyce".

Przykład

```
mount(stage, ctx) {
  const stop = onKey(ctx.signal, (event) => {
    if (event.key === 'a') decide(true);
  });
  ctx.progress(0, 12);
  ctx.finish({ score: 9, max: 12 });
}
```

js/ui.js: budowanie widoku

FUNKCJA `h(tag, props, ...children)`

Tworzy element DOM. Tekst przekazany jako dziecko albo w `text` zawsze trafia przez `textContent`.

Klucze props

Klucz	Opis
`class`	Klasa CSS.
`text`	Treść tekstowa (bezpieczna dla danych od użytkownika).
`html`	Treść HTML. Tylko dla statycznych, autorskich tekstów.
`onclick`, `oninput`, ...	Każdy klucz `on*` z funkcją dodaje nasłuch zdarzenia.
`aria-*`, `data-*`, inne	Ustawiane jako atrybuty. Wartości `null`, `undefined` i `false` są pomijane.

Pozostałe funkcje

Funkcja	Opis
`button(options)`	Przycisk z ikoną i podpowiedzią skrótu. Opcje: `label`, `variant` (`primary`, `secondary`, `ghost`, `danger`), `iconName`, `iconAfter`, `shortcut`, `size` oraz dowolne atrybuty.
`feedback(kind, title, body)`	Komunikat z ikoną. `kind`: `ok`, `bad`, `warn`, `info`.
`rich(markup, tag)`	Element z autorskim HTML.
`onKey(signal, handler)`	Nasłuch klawiatury na dokumencie, ignoruje skróty z Cmd, Ctrl i Alt. Zwraca funkcję zatrzymującą.
`isTyping(event)`	`true`, gdy fokus jest w polu tekstowym. Skróty jednoliterowe muszą to sprawdzać.
`shuffle(lista)`	Nowa, przetasowana kopia tablicy.
`plural(n, jeden, kilka, wiele)`	Polska odmiana liczebników: `plural(22, 'gra', 'gry', 'gier')` zwraca „gry”.
`scrollBehavior()`	`'auto'` przy ograniczonym ruchu, w innym wypadku `'smooth'`.
`clear(el)`	Czyści element i go zwraca.

choiceQuestion · runQuiz · draw

FUNKCJA `runQuiz(container, ctx, questions, options)`

Wyświetla pytania jedno po drugim, z tasowaniem odpowiedzi, skrótami 1 do 4 i Enter, wyjaśnieniami i paskiem postępu. Zwraca `Promise` z liczbą poprawnych odpowiedzi.

Obiekt pytania

Klucz	Wymagany	Opis
`prompt`	tak	Treść pytania (HTML).
`options`	tak	Tablica `{ label, correct, why }`. Dokładnie jedna odpowiedź ma `correct: true`. `why` pojawia się po wybraniu tej odpowiedzi.
`explain`	nie	Wyjaśnienie pokazywane zawsze po odpowiedzi.
`context`	nie	Dodatkowy blok pod pytaniem, np. nagłówki maila albo karta certyfikatu (HTML).
`level`	nie	`'zaawansowany'` = pytanie tylko na poziomie zaawansowanym.
`shuffleOptions`	nie	`false` wyłącza tasowanie odpowiedzi (domyślnie włączone).

Opcje runQuiz

Opcja	Opis
`label` / `title`	Etykieta licznika, np. „Pytanie 3 z 10”.
`offset`, `total`	Gdy quiz jest częścią większej gry: przesunięcie i łączna liczba kroków na pasku postępu.
`onAnswer(pytanie, poprawnie)`	Wywoływane po każdej odpowiedzi. Egzamin zbiera tak tematy do powtórki.

draw(pula, ile, poziom)

Losuje zestaw z puli. Na poziomie podstawowym bierze tylko elementy bez `level`. Na zaawansowanym najpierw wszystkie zaawansowane, potem uzupełnia podstawowymi, a całość tasuje.

```
const questions = draw(POOL, 7, ctx.level);
```

createTerminal

FUNKCJA `createTerminal(options)` js/terminal-engine.js

Symulator terminala Linuksa na wirtualnym systemie plików. Obsługuje potoki `|`, przekierowania `>` i `>>`, wzorce `*` i `?`, cudzysłowy i `\`, uzupełnianie Tab i historię strzałkami.

Opcje

Opcja	Wymagana	Opis
`tree`	tak	Drzewo plików zbudowane z `dir({...})` i `file(treść, tryb, właściciel)`. Domyślny tryb to `-rw-r--r--`, domyślny właściciel to użytkownik.
`user`, `host`	tak	Prompt `user@host`. Katalog domowy to `/home/{user}`.
`intro`	nie	Linie powitalne: tablica `[tekst, klasa]`.
`quick`	nie	Polecenia do wstawienia jednym dotknięciem (ułatwienie na telefonie).
`onRun(state)`	nie	Wywoływane po każdym poleceniu. `state` zawiera `cwd`, `last` (nazwa, flagi, argumenty, `pattern` i `matched` dla grep), `read` (odczytane pliki).
`extraCommands`	nie	Dodatkowe polecenia gry: `{ nazwa: (argv, stdin) => linie }`. Linia to tekst albo `{ text, cls }`. Rzucony `Error` wypisuje się na czerwono.

Polecenia wbudowane

Polecenia	Opis
`pwd` `ls` `cd` `cat`	Poruszanie się i czytanie. `ls -a -l`, `cat ./-nazwa`. Uprawnienia odczytu są sprawdzane.
`grep` `find` `file`	`grep -i -v -c`, `find KATALOG -name -size -type`, rozpoznawanie typu po zawartości.
`sort` `uniq` `wc` `head` `tail`	`sort -r -n`, `uniq -c -u`, `wc -l`, `head/tail -n N`. Działają na plikach i w potokach.
`base64` `chmod` `echo` `mkdir` `touch` `rm`	`base64 -d`, `chmod 600` i `chmod u+r`, zapis przez `>`. `rm` działa tylko w katalogu domowym, a `rm -rf /` jest zablokowane z wyjaśnieniem.
`whoami` `id` `hostname` `uname` `ip` `date` `history` `clear` `man` `sudo`	Informacje i pomoc. `sudo` wyjaśnia, co by zrobiło, ale niczego nie uruchamia.

Przykład

```
const terminal = createTerminal({
  tree: dir({ home: dir({ kursant: dir({ 'flaga.txt': file('FLAGA{…}', '----------')
}) }) }),
  user: 'kursant',
  host: 'kali',
  extraCommands: { ps: () => ['USER PID COMMAND', 'root 1 /sbin/init'] },
  onRun: (state) => checkMission(state)
});
stage.append(terminal.element, terminal.quickBar);
```

Zwracany obiekt

Klucz	Opis
`element`, `quickBar`	Okno terminala i pasek szybkiego wstawiania.
`print(tekst, klasa)`	Wypisuje linie, np. komunikat gry.
`getNode(ścieżka)`, `resolve(ścieżka)`	Dostęp do wirtualnego systemu plików.
`state`, `focus()`	Stan sesji i ustawienie fokusu w polu polecenia.

mountConfigAudit

FUNKCJA `mountConfigAudit(stage, ctx, options)` js/config-audit.js

Panel ustawień z audytem bezpieczeństwa. Używają go gry: router, utwardzanie SSH, raport skanu i audyt prywatności. Za każde ustawienie: 2 punkty za najlepszą wartość, 1 za akceptowalną.

Opcje

Opcja	Wymagana	Opis
`settings`	tak	Tablica ustawień, opis poniżej.
`title`, `subtitle`, `iconName`	tak	Nagłówek panelu.
`okMessage`	tak	Komunikat, gdy wszystko jest bezpieczne.
`preview(values)`	nie	Funkcja zwracająca element podglądu na żywo, np. plik `sshd_config`.

Obiekt ustawienia

Klucz	Opis
`id`, `section`, `label`, `caption`	Identyfikator, grupa w panelu, nazwa, opis pod nazwą.
`options`	Tablica `{ value, label, mono }`, wyświetlana jako przełącznik segmentowy.
`initial`, `best`, `partial`	Wartość startowa, najlepsza i akceptowalna.
`help`, `issue`	„Dlaczego to ważne?” i opis problemu w wyniku audytu.

`settingsToQuestions(settings)` zamienia te same ustawienia na pytania egzaminacyjne.

logViewer

FUNKCJA `logViewer({ title, lines, filters, classify, placeholder })` js/log-viewer.js

Przeglądarka logów z filtrem działającym jak `grep`, podświetlaniem trafień, gotowymi filtrami i przewijaniem dostępnym z klawiatury. Zwraca gotowy panel.

Opcja	Opis
`title`	Ścieżka pliku w nagłówku, np. `/var/log/auth.log`.
`lines`	Tablica linii logu.
`filters`	Przyciski z gotowymi filtrami, np. `['Failed', 'Accepted']`.
`classify(linia)`	Zwraca klasę linii: `log-fail`, `log-ok`, `log-warn`, `log-breach` albo pusty tekst.

playScenario

FUNKCJA `playScenario(stage, ctx, scenario, index, total, { label, nextLabel })`

js/scenario.js

Sytuacja z wyborem reakcji (klawisze A, B, C), punktowana 0, 1 albo 2. Zwraca `Promise` z punktami. Używają jej gry: socjotechnika, incydent, konto.

Klucz scenariusza	Opis
`icon`, `title`, `story`	Ikona, tytuł i opis sytuacji.
`phase`, `time`	Opcjonalnie etap i godzina, np. w reakcji na incydent.
`options`	Tablica `{ label, points, best, why }`. Dokładnie jedna ma `best: true`.
`level`	Opcjonalnie `'zaawansowany'`.

examPool i egzamin

EKSPORT `export function examPool()`

Opcjonalny eksport gry. Zwraca samodzielne pytania w formacie `runQuiz`. Egzamin (`js/games/egzamin.js`) bierze po jednym pytaniu z każdego tematu, uzupełnia losowo do 25 i pomija pytania zaawansowane.

Zasada	Opis
Samodzielność	Pytanie musi być zrozumiałe bez planszy gry. Pytania odwołujące się do konkretnego logu czy nagrania ruchu nie trafiają do puli.
Rejestracja	Nową pulę dopisz do tablicy `TOPICS` w `egzamin.js`: `[tytuł, id gry, funkcja]`.
Próg	80%. Po zaliczeniu odznaka „Egzamin zaliczony”. Wynik trafia do zapisu jak każda inna gra.
Do powtórki	Tematy z błędnymi odpowiedziami pojawiają się jako linki do gier.

Zapis postępu (localStorage)

OBIEKT `localStorage['cyberstart:v1']`

Cały stan użytkownika w jednym kluczu. Każdy odczyt i zapis jest w `try/catch`, więc aplikacja działa także w trybie prywatnym, tylko bez zapamiętywania.

Przykład (dane demo)


```
{
  "results": {
    "terminal": { "score": 24, "max": 24, "xp": 100, "date": "2026-10-
14T09:12:40.118Z" },
    "porty": { "score": 10, "max": 12, "xp": 83, "date": "2026-10-13T18:02:11.904Z"
}
  },
  "theme": "dark",
  "cards": {
    "Phishing": { "box": 3, "due": "2026-10-21" }
  },
  "streak": { "last": "2026-10-14", "count": 4, "best": 6 },
  "daily": { "id": "backup", "done": true, "date": "2026-10-14" },
  "badges": { "pierwszy-krok": "2026-10-11T19:40:02.511Z" },
  "difficulty": "podstawowy"
}
```


Znaczenie kluczy

Klucz	Opis
`results`	Najlepszy wynik każdej gry: punkty, maksimum, XP (0 do 100) i data w ISO 8601.
`theme`	`null` (motyw systemu), `"light"` albo `"dark"`.
`cards`	Fiszki: pudełko 0 do 5 i data następnej powtórki. Odstępy: 0, 1, 3, 7, 14, 30 dni.
`streak`	Seria dni nauki: ostatni dzień, długość, rekord. Liczy się każda ukończona gra.
`daily`	Wyzwanie dnia. Gra dnia wynika z daty, więc jest taka sama przez cały dzień.
`badges`	Zdobyte odznaki z datą.
`difficulty`	Wybrany poziom trudności, wspólny dla wszystkich gier.

Ranga zależy od odsetka ukończonych gier (10, 25, 45, 70 i 100%), więc dodanie nowych gier nie psuje progów.

Eksport i import postępu

PLIK `cyberstart-postep-RRRR-MM-DD.json`

Przyciski w stopce zapisują i wczytują cały stan. Import zastępuje bieżący postęp po potwierdzeniu.

Format pliku

```
{
  "format": "cyberstart-progress",
  "version": 1,
  "exportedAt": "2026-10-14T10:05:12.334Z",
  "data": { … obiekt jak w localStorage … }
}
```

Walidacja przy imporcie

Sytuacja	Wynik
Plik nie jest JSON-em	Komunikat „To nie jest poprawny plik JSON.", stan bez zmian.
Brak `format: "cyberstart-progress"`	Komunikat „Ten plik nie pochodzi z eksportu Cyberstart.", stan bez zmian.
Plik większy niż 512 kB	Odrzucony przed odczytem.
Wynik nieznanej gry albo XP poza 0 do 100	Wpis pominięty.
Odznaka o nazwie spoza wzorca `[a-z0-9-]` albo bez poprawnej daty	Pominięta (chroni m.in. przed wstrzyknięciem HTML).
Fiszka z pudełkiem spoza 0 do 5 albo złą datą	Pominięta.

Odznaki i materiały

PLIK `js/badges.js`

Każda odznaka to `{ id, title, text, icon, test(context) }`. Po każdej grze aplikacja sprawdza wszystkie testy i przyznaje nowe odznaki.

Klucz contextu	Opis
`games`	Lista wszystkich gier, np. do sprawdzenia ukończenia działu.
`terms`	Lista pojęć ze słowniczka.
`outcome`	Wynik bieżącej gry: `xp`, `level`, `daily`, `streak`.

PLIK `js/resources.js`

Zewnętrzne materiały „Kolejny krok w praktyce". `RESOURCES` to słownik `{ label, url, note }`, a `BY_GAME` przypisuje je do gier. Każdy adres trzeba sprawdzić ręcznie, zanim trafi do listy. Linki otwierają się w nowej karcie z `rel="noopener noreferrer"`.

PWA i service worker

Element	Opis
`manifest.webmanifest`	Nazwa, kolory, ikony 192 i 512 px (maskowalne) oraz SVG. Pozwala zainstalować aplikację na telefonie.
`sw.js`	Najpierw sieć z rewalidacją (`cache: 'no-cache'`), a przy braku sieci pamięć podręczna. Online użytkownik zawsze dostaje aktualne pliki, offline ostatnią wersję.
`CACHE`	Nazwa pamięci, np. `cyberstart-v2` . Zmieniaj ją przy każdym wdrożeniu, żeby stare wpisy zostały usunięte.
Rejestracja	W `js/app.js` , tylko po HTTP lub HTTPS, po zdarzeniu `load` .

Wdrożenie

Aplikacja działa na każdym hostingu statycznym (GitHub Pages, Cloudflare Pages, Netlify). Przed publikacją uzupełnij pola `[[...]]` w `polityka-prywatnosci.html` , `regulamin.html` , `robots.txt` i `sitemap.xml` oraz ustaw na hostingu nagłówki bezpieczeństwa (CSP, HSTS, `X-Content-Type-Options` , `Referrer-Policy`).

Testy e2e

KOMENDA `npm run test:e2e`

Playwright uruchamia serwer na porcie 4181 i dwa projekty: `desktop` (1280 px) i `mobile` (375 px). Pełny przebieg trwa około 40 sekund.

Plik	Co sprawdza
`games.spec.js`	Każda gra da się przejść do ekranu wyniku, na obu poziomach, bez błędów w konsoli.
`a11y.spec.js`	axe-core (WCAG 2.2 AA) na każdym ekranie, w trakcie gry i na ekranie wyniku, w obu motywach.
`mobile.spec.js`	Brak poziomego przewijania na wszystkich etapach gier, cele dotyku co najmniej 44 px.
`robustness.spec.js`	Silnik terminala, obsługa klawiaturą, tryb offline, zmanipulowany import, brak błędów na wszystkich trasach.
`exam.spec.js`	Zaliczenie egzaminu z odznaką oraz niezaliczony egzamin z listą tematów do powtórki i działającymi linkami.
`mechanics.spec.js`	Wyzwanie dnia, seria, odznaki, linki do praktyki, service worker, ponowna gra, zapamiętany poziom.
`app.spec.js`, `interactions.spec.js`	Mapa, tytuły stron, 404, pliki obowiązkowe, motyw, eksport i import, przeciąganie, skróty.

Autoplayer

`tests/e2e/helpers.js` zawiera funkcję, która przechodzi grę bez znajomości odpowiedzi: klika odpowiedzi, przyciski „Dalej” i obsługuje nietypowe mechaniki w funkcji `special()` (terminal, uprawnienia, dopasowywanie, układanie, CTF). Opcja `stopAfter` zatrzymuje grę w trakcie, a `resume` kontynuuje, dzięki czemu testy sprawdzają też stany pośrednie.